

AI Kids Video Generator

Complete Technical Reference & Roadmap

GitHub: [omidgh1/AI-Animation](#) | Branch: dev | May 2026

~\$0.50 cost per video	9 stages fully automated	~3 min per video	YouTube Shorts ready
----------------------------------	------------------------------------	----------------------------	--------------------------------

This document covers the complete architecture, every pipeline stage, all bugs fixed, the full-run workflow, YouTube auto-upload (Stage 10), and the long-video expansion plan.

1. Project Overview

A fully automated Python pipeline that turns a one-line topic into a publish-ready kids video in under 5 minutes, for approximately \$0.50. No manual editing required. The pipeline runs entirely on your local Windows machine (D:\AI-Animation) using cloud AI APIs for generation and FFmpeg for rendering.

Tech Stack

Component	Tool / API	Purpose	Cost
Story & Refinement	Claude claude-sonnet-4-5 (Anthropic)	12-scene script + art direction	~\$0.02
Image Generation	Fal.ai Flux Pro v1.1	12 PNG images @ 1080x1920	\$0.48
Voice Narration	ElevenLabs	Per-scene MP3 clips + full narration	\$0.00 (free tier)
Background Music	Bensound.com	Royalty-free MP3 tracks	\$0.00
Subtitle Export	Script-based (no API)	SRT for YouTube + ASS for burn-in	\$0.00
Video Render	FFmpeg (local)	Ken Burns + audio replace + subtitles	\$0.00
Thumbnail	Fal.ai Flux Pro v1.1	2 thumbnails + Pillow text overlay	~\$0.08
YouTube Upload	YouTube Data API v3	Auto-upload + metadata (Stage 10)	\$0.00

Project Folder Structure

Root: D:\AI-Animation\ | Branch: dev | Python: 3.13 | Venv: .venv

```
D:\AI-Animation\ ■■■ test.py ← main entry point (set TEST_STAGE + LOAD_EXISTING_SLUG) ■■■
config.py ← all settings, API keys, paths ■■■ requirements.txt ■■■ .env ← API keys (never
commit) ■■■ .env.example ← template ■■■ add_thumbnail_text.py ← deprecated (now
integrated into Stage 9) ■■■ discover_voices.py ■■■ automation/ ■■■ batch_topics.txt ←
30 ready-to-use story topics ■■■ output/ ■■■ stories/ ← .json + _refined.json ■■■
raw// ← scene_01.png ... scene_12.png + manifest.json ■■■ audio// ← scene_NN.mp3 +
full_narration.mp3 + final_mix.mp3 ■■■ music// ← background_playful.mp3 ■■■
subtitles// ← .srt + .ass ■■■ final// ← _final.mp4 + render_manifest.json ■■■
thumbnails// ← thumbnail_16x9.png + thumbnail_9x16.png (+ _text versions) ■■■ pipeline/
■■■ models.py ← Pydantic: VideoScript, RefinedScript, RefinedScene ■■■ story_generator.py
← Stage 1 ■■■ scene_refiner.py ← Stage 2 ■■■ image_generator.py ← Stage 3 ■■■
voice_generator.py ← Stage 4 ■■■ music_handler.py ← Stage 5 ■■■ subtitle_generator.py ←
Stage 7 ■■■ video_renderer.py ← Stage 8 ■■■ thumbnail_generator.py ← Stage 9
```

2. API Keys & Configuration

Add these to your .env file in the project root:

```
# Required ANTHROPIC_API_KEY=sk-ant-... # console.anthropic.com FAL_KEY=key-... #
fal.ai/dashboard/keys ELEVENLABS_API_KEY=... # elevenlabs.io # Optional PIXABAY_API_KEY=...
# pixabay.com/api/docs/ (free) # (not used - Bensound is the music source)
```

Key Configuration Settings (config.py)

Setting	Value	Purpose
STORY_MODEL	claude-sonnet-4-5	Claude model for Stages 1 & 2
STORY_MAX_TOKENS	4096	Stage 1 output budget
REFINE_MAX_TOKENS	8192	Stage 2 output budget (2 passes x 6 scenes)
FAL_IMAGE_MODEL	fal-ai/flux-pro/v1.1	Primary image model
FAL_IMAGE_SIZE	portrait_16_9	9:16 vertical for Shorts
ELEVENLABS_MODEL	eleven_multilingual_v2	Voice model
MUSIC_VOLUME	0.20	Background music level (20%)
NUM_SCENES	12	Scenes per short video
FFMPEG_BIN	C:\Users\omidg\AppData\...\bin	FFmpeg executable path

3. Pipeline Stages (Shorts — 60 seconds)

Stage 1 — Story Generation

File: pipeline/story_generator.py

Calls Claude claude-sonnet-4-5 with a one-line topic and returns a validated 12-scene VideoScript JSON. The system prompt enforces the emotional arc (hook → problem → helper → solution → celebration), character description, camera movements, emotions, and sound effects. A `_normalize()` method auto-repairs missing fields before Pydantic validation so the pipeline never crashes on Claude output variation.

Input: TEST_TOPIC string

Output: output/stories/.json

Cost: ~\$0.01 Claude API

Notes: Emotion per scene is now explicitly set in the system prompt: S1=surprised, S2=happy, S3=scared, S4=sad, S5=sad, S6=surprised, S7=curious, S8=excited, S9=excited, S10=excited, S11=proud, S12=calm. This drives ElevenLabs voice style selection in Stage 4.

Stage 2 — Scene Refinement

File: pipeline/scene_refiner.py

Calls Claude a second time, this time as an art director. Takes the Stage 1 script and produces enhanced image prompts (80-140 words each with foreground/mid/background depth and lighting), punchy on_screen_text (1-4 emotional words like 'OH NO!' or 'YOU DID IT!'), negative_prompt per scene, colour_palette (3 hex codes), global_style_suffix, thumbnail_concept, and colour_story.

Input: output/stories/.json

Output: output/stories/_refined.json

Cost: ~\$0.02 Claude API

Notes: Uses a two-pass split strategy to avoid max_token truncation: Pass A refines scenes 1-6 + global fields, Pass B refines scenes 7-12. Results are merged into a single RefinedScript. REFINE_MAX_TOKENS=8192 per pass.

Stage 3 — Image Generation

File: pipeline/image_generator.py

Generates all 12 scene images in parallel using Fal.ai Flux Pro v1.1. Uses asyncio with a semaphore limiting concurrency to 4 simultaneous requests. Each image is portrait_16_9 (1080x1920). Falls back to fal-ai/flux/dev automatically if the primary model fails on any scene.

Input: output/stories/_refined.json

Output: output/raw//scene_01.png ... scene_12.png + manifest.json

Cost: \$0.48 (12 x \$0.04 Fal.ai)

Notes: manifest.json records file path, size_kb, elapsed time, model used, and image URL per scene. Cached images are reused — Stage 3 skips scenes that already have valid PNG files on disk.

Stage 4 — Voice Generation

File: pipeline/voice_generator.py

Generates per-scene narration MP3 clips using ElevenLabs. Voice selection is driven by the scene emotion field: warm/calm voices for happy/calm scenes, energetic voices for excited scenes, softer voices for sad/scared scenes. Also produces full_narration.mp3 for subtitle timing.

Input: output/stories/_refined.json

Output: output/audio//scene_NN.mp3 + full_narration.mp3 + voice_manifest.json

Cost: \$0.00 (ElevenLabs free tier)

Notes: Python 3.13 removed audioop — install audioop-libs to restore pydub compatibility. Free premade voices: Matilda (XrExE9yKlg1WjnnIVkGX), Jessica (cgSgspJ2msm6clMCkdW9), Clara (2OEeJcYw2f3bWMzzjVMU).

Stage 5 — Background Music

File: pipeline/music_handler.py

Downloads a royalty-free MP3 from Bensound.com matched to the story's background_music_mood field. Mixes narration + music using pydub with MUSIC_VOLUME=0.20 (20% music level). Outputs final_mix.mp3 which Stage 8 uses as the sole audio track for the video.

Input: output/audio//full_narration.mp3

Output: output/audio//final_mix.mp3 + output/music//background_.mp3

Cost: \$0.00 (Bensound free with attribution)

Notes: Add 'Music by Bensound.com' to your YouTube description. Available moods: playful, adventurous, magical, calm_and_soothing, heartwarming, funny, dramatic. Falls back to pydub-generated silence if download fails.

Stage 6 — Sound Effects

Not yet implemented. The story JSON includes sound_effect cues per scene (name, timing, volume) but they are not yet played in the final render. Future: integrate Freesound.org API or Pixabay audio for per-cue SFX injection.

Stage 8 — Video Render (run before Stage 7)

File: pipeline/video_renderer.py

Assembles the final 9:16 MP4 using FFmpeg. For each scene: applies Ken Burns pan/zoom effect to the static image (matching camera_movement field), adds the scene's narration audio. Concatenates all 12 clips. Replaces the concatenated audio with final_mix.mp3 (narration + music) using -map 0:v -map 1:a to avoid echo. Optionally burns ASS subtitles.

Input: output/raw// + output/audio//final_mix.mp3 + output/subtitles//.ass

Output: output/final//_final.mp4 + render_manifest.json

Cost: \$0.00 (FFmpeg local)

Notes: CRITICAL: Run Stage 8 BEFORE Stage 7. render_manifest.json records the exact frame-accurate duration per scene that FFmpeg actually produced. Stage 7 reads this for perfect subtitle sync. Audio replacement (-map 1:a) was the fix for the echo bug caused by amix doubling the narration.

Stage 7 — SRT Subtitle Export (run AFTER Stage 8)

File: pipeline/subtitle_generator.py

Generates .SRT (upload to YouTube) and .ASS (burn into video) subtitle files. Reads render_manifest.json for frame-accurate scene timings. SRT contains full narration text split across scenes. ASS uses kids-style formatting: Arial Rounded, size 72, white text with black outline, colour cycling.

Input: output/final//render_manifest.json

Output: output/subtitles//.srt + .ass

Cost: \$0.00 (no API needed)

Notes: Upload the .SRT file to YouTube Studio alongside your video for CC captions and improved SEO. The .ASS file is for the burned-in subtitle text on the video itself (optional — Stage 8 handles this).

Stage 9 — Thumbnail Generation

File: pipeline/thumbnail_generator.py

Generates 2 thumbnail images using Fal.ai Flux: landscape_16_9 (1280x720) for YouTube and portrait_16_9 (1080x1920) for Shorts. Then adds bold title text overlay using Pillow: white Arial Bold with black outline, auto-generated from the story title using build_short_title(). Outputs 4 files — raw and _text versions of each.

Input: output/stories/_refined.json

Output: output/thumbnails//thumbnail_16x9.png + thumbnail_9x16.png + _text versions

Cost: ~\$0.08 (2 x \$0.04 Fal.ai)

Notes: Upload thumbnail_16x9_text.png to YouTube Studio as the custom thumbnail. Use thumbnail_9x16_text.png as the Shorts cover image. Cached thumbnails are reused — re-running Stage 9 only re-applies the text overlay (no extra charge).

4. Full-Run Workflow (One Video — End to End)

Set these values at the top of test.py, then run the stages in order. After Stage 1 completes, copy the slug from the output and paste it into LOAD_EXISTING_SLUG for all subsequent stages — this reuses saved outputs and avoids re-running expensive stages.

Step 1 — Configure test.py

```
# Set once at the top of test.py: TEST_TOPIC = "Titanic story for kids" # your one-line idea
TEST_STAGE = 1 # change this each run LOAD_EXISTING_SLUG = None # set after Stage 1
completes
```

Step 2 — Run each stage in order

Stage	Set TEST_STAGE to	Set LOAD_EXISTING_SLUG to	Time	Cost
1 — Story	1	None (generate fresh)	~10s	\$0.01
2 — Refinement	2	your-story-slug	~50s (2 Claude calls)	\$0.02
3 — Images	3	your-story-slug	~60s (parallel)	\$0.48
4 — Voice	4	your-story-slug	~40s	\$0.00
5 — Music	5	your-story-slug	~5s (download)	\$0.00
8 — Render video	8	your-story-slug	~90s (FFmpeg)	\$0.00
7 — SRT subtitles	7	your-story-slug	~1s	\$0.00
9 — Thumbnails	9	your-story-slug	~30s	\$0.08
10 — Upload	10	your-story-slug	~30s	\$0.00

Total per video: ~6 minutes wall time, ~\$0.59 in API costs.

Step 3 — Upload to YouTube

- Go to **studio.youtube.com** → Create → Upload video
- Upload **output/final/<slug>/<slug>_final.mp4**
- Title: copy from the story JSON (refined.title)
- Description: copy from refined.youtube_description — add 'Music by Bensound.com' at the bottom
- Tags: copy from refined.youtube_tags
- Thumbnail: upload **output/thumbnails/<slug>/thumbnail_16x9_text.png**
- Subtitles: upload **output/subtitles/<slug>/<slug>.srt** under Subtitles tab
- Set audience: Yes, this is made for kids
- Publish or schedule

5. Stage 10 — YouTube Auto-Upload (Next to Build)

Stage 10 will use the YouTube Data API v3 to fully automate the upload process. This is free — YouTube's API costs nothing. You need to create OAuth2 credentials once.

5.1 One-time Setup

- Go to **console.cloud.google.com** → New Project
- Enable the **YouTube Data API v3**
- Create OAuth 2.0 credentials → Desktop App → download credentials.json → save to D:\AI-Animation\
- First run will open a browser for Google login — saves token.json for future runs
- Install: **pip install google-api-python-client google-auth-oauthlib**

5.2 What Stage 10 Will Do

```
# pipeline/youtube_uploader.py – Stage 10 plan class YouTubeUploader: def upload(self, refined: RefinedScript) -> str: # 1. Authenticate with OAuth2 (token.json cached after first login) # 2. Upload the video file with title, description, tags, category # 3. Set the custom thumbnail (thumbnail_16x9_text.png) # 4. Upload the SRT subtitle file # 5. Return the YouTube video URL pass # Output: https://youtube.com/shorts/VIDEO_ID
```

5.3 Key API Calls

Action	API Method	Notes
Upload video	youtube.videos().insert()	multipart upload, resumable for large files
Set thumbnail	youtube.thumbnails().set()	upload thumbnail_16x9_text.png
Upload subtitles	youtube.captions().insert()	upload .srt with language='en'
Schedule publish	publishAt in snippet	set future datetime in ISO 8601 format

5.4 Required Metadata (all auto-generated from pipeline)

```
metadata = { "title": refined.title, # from Stage 1 "description": refined.youtube_description # from Stage 1 + "\n\nMusic by Bensound.com", "tags": refined.youtube_tags, # from Stage 1 "categoryId": "22", # People & Blogs (or 1 = Film) "madeForKids": True, "privacyStatus": "public" # or "private" to review first }
```

5.5 test.py Integration

```
# In test.py, Stage 10 will be: TEST_STAGE = 10 LOAD_EXISTING_SLUG = "timmys-big-ship-adventure" # It will: # - Load the refined script # - Find the final video, thumbnail, and SRT files # - Upload everything to YouTube automatically # - Print the live YouTube URL when done
```

Quota note: YouTube Data API gives you 10,000 units/day free. One video upload costs ~1,600 units. That's 6 free uploads per day with a standard quota. Apply for a quota increase in Google Cloud Console if you need more.

6. Bugs Fixed During Development

All scenes emotion='happy'

System prompt did not specify emotion per scene position. Claude defaulted all to 'happy'. Fixed by explicitly listing the expected emotion for each scene slot in the SYSTEM_PROMPT.

Fix location: *story_generator.py* — *SYSTEM_PROMPT*

Stage 2 JSON truncation

12 scenes x 140 word prompts = ~3000 tokens output, hitting the 4096 max_tokens limit mid-scene. Fixed with a two-pass split strategy: Pass A handles scenes 1-6 + global fields, Pass B handles 7-12.

REFINE_MAX_TOKENS raised to 8192.

Fix location: *scene_refiner.py* — *refine()* + *_merge_passes()*

Echo in final video

Stage 8 was using amix to blend audio streams. But final_mix.mp3 already contained the narration, so it played twice. Fixed by replacing the video audio entirely with -map 0:v -map 1:a instead of amix.

Fix location: *video_renderer.py* — *render()*

Subtitle timing mismatch

Stage 7 was using voice manifest estimated durations, which differed from actual FFmpeg-rendered clip lengths. Fixed by reading render_manifest.json (produced by Stage 8) for frame-accurate scene timings. Stage 7 must now always run AFTER Stage 8.

Fix location: *subtitle_generator.py* — *_get_scene_timings()*

ModuleNotFoundError: pyaudioop / audioop

Python 3.13 removed the audioop module. pydub depends on it. Fixed by installing the audioop-Its drop-in replacement: pip install audioop-Its

Fix location: *requirements.txt*

animation_generator import error (Stage 8)

video_renderer.py still imported the deleted animation_generator module from the old animation pipeline. Fixed by rewriting _get_scene_files() to read directly from output/raw/ (Stage 3 PNG images).

Fix location: *video_renderer.py* — *_get_scene_files()*

Bensound / Pixabay music download failure

Hardcoded CDN URLs became stale. Pixabay's music API is web-only (not programmatic). Switched to Bensound.com stable direct MP3 URLs with a pydub silence fallback.

Fix location: *music_handler.py* — *download_music()*

Thumbnail text clipped (portrait)

Font size 110px overflowed the 1080px wide portrait canvas. Fixed: portrait font=75px, y_percent=0.85; landscape font=95px, y_percent=0.78.

Fix location: *thumbnail_generator.py* — *add_title_text()*

on_screen_text mechanical slicing

Stage 1 was slicing the narration text for on_screen_text (e.g. 'Timmy the teddy bear!'). Fixed in Stage 2 system prompt which rewrites it to punchy 1-4 word emotional peaks.

Fix location: *scene_refiner.py* — *SYSTEM_PROMPT*

7. Cost Breakdown & Business Model

Per Video (60-second Short)

Stage	Service	Cost/Video
Stage 1+2 — Story & Refinement	Anthropic Claude API	~\$0.02
Stage 3 — Image Generation (12 images)	Fal.ai Flux Pro v1.1	\$0.48
Stage 4 — Voice Narration	ElevenLabs (free tier)	\$0.00
Stage 5 — Background Music	Bensound.com (free + attribution)	\$0.00
Stage 7+8 — Subtitles + Render	Local FFmpeg	\$0.00
Stage 9 — Thumbnails (2 images)	Fal.ai Flux Pro v1.1	\$0.08
Stage 10 — YouTube Upload	YouTube Data API (free)	\$0.00
TOTAL		~\$0.58

YouTube Revenue Projections

Scenario	CPM (per 1000 views)	Break-even views
YouTube Kids (conservative)	\$0.50	~1,160 views
YouTube Kids (average)	\$1.50	~387 views
YouTube Kids (good month)	\$3.00	~193 views
Long-form (8+ min)	\$3-8 CPM	~73-193 views

Target: 30 videos/month = ~\$17.40/month in production costs. At 1,000 views/video average, that's 30,000 views/month = ~\$45-\$90 revenue. Channel breaks even after the first few hundred views per video.

8. Long Video Expansion Plan (5-15 minutes)

Long-form YouTube videos (8+ minutes) unlock mid-roll ads, significantly higher CPM (\$3-8), and better discoverability. The same pipeline can produce them by scaling the scene count. No new APIs needed — just config changes.

8.1 Architecture Changes

Parameter	Shorts (current)	5 min video	10 min video	15 min video
NUM_SCENES	12	30	60	90
SCENE_DURATION	5s avg	10s avg	10s avg	10s avg
Image cost	\$0.48	\$1.20	\$2.40	\$3.60
Total cost	~\$0.58	~\$1.30	~\$2.50	~\$3.70
Stage 2 passes	2 (6+6)	5 (6 scenes each)	10 passes	15 passes
Render time	~90s	~4 min	~8 min	~12 min

8.2 Config Changes Required

```
# config.py – change these for long video mode: NUM_SCENES = 60 # or 30, 90 depending on
target length SCENE_DURATION = 10 # seconds per scene (vs 5 for Shorts) # The rest of the
pipeline adapts automatically: # - Stage 1: generates NUM_SCENES scenes # - Stage 2:
auto-splits into ceil(NUM_SCENES/6) passes # - Stage 3: generates NUM_SCENES images in
parallel # - Stage 4: generates NUM_SCENES voice clips # - Stage 8: renders NUM_SCENES clips
into one video
```

8.3 Story Structure for Long Videos

The 12-scene structure maps to a 3-act story. For long videos, expand each act proportionally:

Act	Short (12 scenes)	Long 60-scene version	Purpose
Act 1: Setup	Scenes 1-3	Scenes 1-12	World, character, normal life
Act 2: Adventure	Scenes 4-9	Scenes 13-48	Problem, attempts, helpers, teamwork
Act 3: Resolution	Scenes 10-12	Scenes 49-60	Breakthrough, celebration, lesson, ending

8.4 Story Generator Changes for Long Videos

```
# In SYSTEM_PROMPT (story_generator.py), add this for long videos:
LONG_VIDEO_SCENE_STRUCTURE = """ SCENE STRUCTURE ({num_scenes} scenes total for
{target_minutes} minute video): ACT 1 – WORLD BUILDING (scenes 1-{act1_end}): - Scene 01:
HOOK – Start mid-action, something surprising - Scenes 02-04: World – show the character's
happy normal life in detail - Scenes 05-{act1_end}: Setup – introduce supporting characters,
the world, daily routines ACT 2 – ADVENTURE (scenes {act2_start}-{act2_end}): - Problem
appears and escalates through multiple attempts - Different helpers arrive, each
contributing something - Teamwork and learning moments interspersed - Near-miss moments that
raise stakes ACT 3 – RESOLUTION (scenes {act3_start}-{num_scenes}): - Breakthrough moment -
Celebration with all characters - Lesson shared - Warm goodbye with callback to the opening
"""
```

8.5 Stage 2 Multi-Pass for Long Videos

```
# scene_refiner.py already handles this automatically. # For 60 scenes: ceil(60/6) = 10
passes # Each pass handles 6 scenes to stay within token limits. # The merge step combines
all passes into one RefinedScript. # The refine() method already uses: # script_a =
_script_with_scenes(script, script.scenes[i*6:(i+1)*6]) # This naturally extends to any
number of scenes.
```

8.6 YouTube Optimization for Long Videos

- Set video duration to at least **8 minutes** to unlock mid-roll ads
- Use chapters in description: timestamps every 2-3 minutes for better retention
- Long videos have better SEO — YouTube indexes more content
- Target search keywords in title and description, not just hashtags
- Recommended titles: 'The Complete Story of...' / 'Full Episode:...' / '12 Minutes of...'
- Long-form kids content CPM: \$3-8 vs Shorts \$0.03-0.08 — 40-100x better monetisation

9. Batch Automation & Scaling

9.1 automation/batch_topics.txt

The file automation/batch_topics.txt contains 30 pre-written topics. Format: one topic per line, optionally with | category:

```
# automation/batch_topics.txt A tiny elephant who is afraid of water | animal_adventure Two kittens who learn to share | friendship_and_emotions A magic paintbrush that brings drawings to life | magic_and_fantasy # ... 27 more topics
```

9.2 Batch Run Script (Future: run_batch.py)

```
# Future automation/run_batch.py # Run this to produce and upload one video per topic automatically from pipeline
from pipeline.story_generator import StoryGenerator
from pipeline.scene_refiner import SceneRefiner
from pipeline.image_generator import ImageGenerator
from pipeline.voice_generator import VoiceGenerator
from pipeline.music_handler import MusicHandler
from pipeline.video_renderer import VideoRenderer
from pipeline.subtitle_generator import SubtitleGenerator
from pipeline.thumbnail_generator import ThumbnailGenerator
# from pipeline.youtube_uploader import YouTubeUploader

# Stage 10
def run_full_pipeline(topic: str, category: str = None):
    # Stage 1 script = StoryGenerator().generate(topic, category)
    # Stage 2 refined = SceneRefiner().refine(script)
    # Stage 3 ImageGenerator().generate(refined)
    # Stage 4 VoiceGenerator().generate(refined)
    # Stage 5 MusicHandler().create_mix(refined)
    # Stage 8 (before 7!) VideoRenderer().render(refined, burn_subtitles=True)
    # Stage 7 SubtitleGenerator().generate(refined)
    # Stage 9 ThumbnailGenerator().generate(refined)
    # Stage 10 (when ready)
    # YouTubeUploader().upload(refined)
    if __name__ == "__main__":
        topics = open("automation/batch_topics.txt").readlines()
        for line in topics:
            line = line.strip()
            if not line or line.startswith("#"):
                continue
            parts = line.split("|")
            topic = parts[0].strip()
            category = parts[1].strip() if len(parts) > 1 else None
            print(f"\nProcessing: {topic}")
            run_full_pipeline(topic, category)
```

9.3 Tested Videos

Video	Slug	Status	Notes
Spark's Big Brave Moment	sparks-big-brave-moment	Complete	First test video — tiny purple dragon
Timmy's Big Ship Adventure	timmys-big-ship-adventure	Complete	All 9 stages. Teddy bear + dolphin Dolly. Bathtub
Stella's Big Ship Adventure	stellas-big-ship-adventure	Stages 1+2	Bunny in toy ship. Images not yet generated.

10. Quick Reference Card

test.py — The Only File You Edit

```
# test.py – edit ONLY these lines at the top: TEST_STAGE = 1 # 1,2,3,4,5,8,7,9,10 – run in
this order! TEST_TOPIC = "a tiny dragon afraid of fire" TEST_CATEGORY = None # None=auto,
or: "animal_adventure", # "friendship_and_emotions", "magic_and_fantasy", #
"learning_and_educational", "bedtime_and_calming" LOAD_EXISTING_SLUG = None # e.g.
"timmys-big-ship-adventure" (after Stage 1) PRINT_JSON = False # True = print raw JSON
SAVE_OUTPUT = True
```

Stage Order (CRITICAL)

Always run in this order. Stage 7 MUST come after Stage 8:

1 → 2 → 3 → 4 → 5 → 8 → 7 → 9 → 10

Common Commands

```
# Install dependencies (Windows PowerShell) cd D:\AI-Animation .venv\Scripts\activate pip
install -r requirements.txt pip install audioop-lts # Python 3.13 fix for pydub # Run the
pipeline python test.py # then change TEST_STAGE and repeat # Push to GitHub git add -A git
commit -m "Add new video: topic name" git push origin dev
```

Voice IDs (ElevenLabs free tier)

Voice Name	Voice ID	Best for
Matilda (warm narrator)	XrExE9yKIg1WjnnIVkGX	happy, calm, proud, magical scenes
Jessica (energetic)	cgSgspJ2msm6clMCkdW9	excited, surprised, curious scenes
Clara (soft/gentle)	2OEeJcYw2f3bWMzzjVMU	sad, scared scenes

Useful URLs

- **Anthropic Console:** console.anthropic.com
- **Fal.ai Dashboard:** fal.ai/dashboard
- **ElevenLabs:** elevenlabs.io
- **Bensound Music:** bensound.com
- **YouTube Studio:** studio.youtube.com
- **Google Cloud Console:** console.cloud.google.com
- **GitHub Repo:** github.com/omidgh1/AI-Animation (branch: dev)